Machine Learning Student Notes

XGBoost Regressor & Gradio

A simple English guide for understanding regression with XGBoost and building a user interface with Gradio.

Includes: beginner-friendly concepts, an eye-medical example, XGBoost parameters, regression metrics, Gradio components, and a complete model-to-interface workflow.

For classroom and lab use. The medical example is educational and is not intended for diagnosis or clinical decision-making.

1. What is Regression?

Regression is a supervised machine learning task used when the target we want to predict is a **continuous numerical value**.

Regression examples	Classification examples
House price Blood glucose level Hospital stay duration Medical cost Intraocular Pressure (IOP)	Glaucoma: Yes / No Diabetes: Yes / No Tumor: Benign / Malignant Disease severity category

Simple rule: If the output is a number such as 18.6, 42.3, or 105.7, regression is usually appropriate. If the output is a class or category, classification is usually appropriate.

2. What is XGBoost Regressor?

XGBoost stands for **Extreme Gradient Boosting**. XGBRegressor is the XGBoost model used for regression problems.

Student definition: XGBoost Regressor builds many decision trees one after another. Each new tree tries to correct the mistakes made by the previous trees so that the final numerical prediction becomes more accurate.

The core idea of boosting

Tree 1 -> First Prediction -> Error

|

Tree 2 -> Corrects part of the error

|

Tree 3 -> Corrects more error

|

Final improved prediction

3. Eye-Medical Example

Suppose we want to predict a patient's **Intraocular Pressure (IOP)** using eye and patient measurements.

Feature	Meaning	Example
Age	Patient age	55 years
Corneal Thickness	Thickness of the cornea	535 micrometers
Cup-to-Disc Ratio	Eye measurement used in glaucoma assessment	0.55
Visual Field MD	Visual field mean deviation	-4.5 dB
Blood Pressure	Systolic / diastolic blood pressure	135/85
Family History	Whether glaucoma is present in family history	1 = Yes
Target	Intraocular Pressure	20.0 mmHg

How sequential correction works

Actual IOP = 20 mmHg. Tree 1 predicts 16 mmHg, so the error is 4. Tree 2 may add a correction of +2.5, giving 18.5. Tree 3 then makes another smaller correction, perhaps producing 19.5 mmHg. The error becomes much smaller.

What does “gradient” mean?

For beginners, think of the gradient as information that tells the model **which direction the prediction should move to reduce the error**. If the prediction is too low, it should move upward; if it is too high, it should move downward.

What does “boosting” mean?

Boosting combines several weak or simple models to create a stronger model. In XGBoost, trees are added **sequentially**, so later trees focus on correcting earlier errors.

Useful comparison: Random Forest mainly builds many trees independently and combines them. XGBoost builds trees sequentially so later trees learn from earlier mistakes.

4. Basic XGBoost Regression Code

```
from xgboost import XGBRegressor

model = XGBRegressor()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Three commands students should remember

Command	Meaning
model = XGBRegressor()	Create the regression model.
model.fit(X_train, y_train)	Train the model using training data.
model.predict(X_test)	Predict numerical target values for unseen data.

Main parameters

```
model = XGBRegressor(
    n_estimators=200,
    max_depth=3,
    learning_rate=0.05,
    subsample=0.9,
    colsample_bytree=0.9,
    random_state=42
)
```

Parameter	Simple meaning
n_estimators	Number of boosting trees.
max_depth	Maximum complexity or depth of each tree.
learning_rate	How strongly each new tree changes the model. Smaller values mean slower, more careful learning.
subsample	Fraction of training rows used by each tree.
colsample_bytree	Fraction of features available to each tree.
random_state	Helps make results reproducible.

5. Training and Evaluation

Train/Test split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)
```

A common beginner setup is **80% training data** and **20% testing data**. The model learns from the training set and is evaluated on unseen test data.

Regression metrics

Metric	Meaning	Better value
MAE	Mean Absolute Error: average absolute difference between actual and predicted values.	Lower
RMSE	Root Mean Squared Error: measures prediction error and penalizes large mistakes more strongly.	Lower
R2	Coefficient of Determination: how much variation in the target is explained by the model.	Closer to 1

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print("MAE:", mae)
print("RMSE:", rmse)
print("R2:", r2)
```

Feature importance

XGBoost can provide feature importance values showing which variables were more useful to the model for prediction.

Important: Feature importance does not automatically prove medical causation. It only shows that the model found a feature useful for prediction.

6. What is Gradio?

Gradio is a Python library used to quickly build a simple web-based user interface for a Python function or machine learning model.

Student definition: Gradio collects input from a user, sends the input to a Python function or machine learning model, and then displays the result in a web interface.

Why use Gradio?

Without Gradio, a programmer may run prediction code directly:

```
prediction = model.predict(new_patient)
print(prediction)
```

A normal user should not need to edit Python code. Gradio can provide input boxes, sliders, radio buttons, a prediction button, and an output area.

User -> Gradio Inputs -> Python Function -> ML Model
Prediction Result -> Gradio Output -> User

XGBoost = Brain Performs the machine learning prediction.	**Gradio = Interface** Allows a user to interact with the model.

Very simple Gradio example

```
!pip install gradio

import gradio as gr

def greet(name):
    return "Hello " + name

app = gr.Interface(
    fn=greet,
    inputs="text",
    outputs="text"
)

app.launch()
```

7. Understanding gr.Interface()

```
app = gr.Interface(
    fn=greet,
    inputs="text",
    outputs="text"
)
```

Part	Meaning
fn=greet	The Python function that Gradio should run.
inputs="text"	The user enters text.
outputs="text"	The result is displayed as text.
app.launch()	Starts the Gradio application.

Common Gradio components

Component	Use
gr.Number()	Numeric input such as age or blood pressure.
gr.Textbox()	Text input.
gr.Slider()	Choose a value within a range.
gr.Radio()	Select one option, such as Yes/No.
gr.Button()	Run an action when clicked.

Interface vs Blocks

Gradio Interface	Gradio Blocks
Fast and beginner-friendly	More flexible and customizable
Good for simple demos	Good for dashboards and advanced layouts
Less UI control	Supports rows, columns, tabs, buttons, and more

8. Connecting Gradio with XGBoost

The trained XGBoost model can be placed inside a prediction function. Gradio sends the user's values to this function.

```
def predict_iop(age, corneal_thickness, cup_disc_ratio,
                visual_field, systolic_bp, diastolic_bp,
                corneal_curvature, family_history):

    patient = pd.DataFrame({
        "age": [age],
        "corneal_thickness_um": [corneal_thickness],
        "cup_disc_ratio": [cup_disc_ratio],
        "visual_field_md_db": [visual_field],
        "systolic_bp": [systolic_bp],
        "diastolic_bp": [diastolic_bp],
        "corneal_curvature_d": [corneal_curvature],
        "family_history_glaucoma": [family_history]
    })

    prediction = model.predict(patient)[0]
    return round(prediction, 2)
```

Gradio interface for the prediction function

```
app = gr.Interface(
    fn=predict_iop,
    inputs=[
        gr.Number(label="Age"),
        gr.Number(label="Corneal Thickness"),
        gr.Number(label="Cup Disc Ratio"),
        gr.Number(label="Visual Field MD"),
        gr.Number(label="Systolic BP"),
        gr.Number(label="Diastolic BP"),
        gr.Number(label="Corneal Curvature"),
        gr.Radio(choices=[0, 1], label="Family History")
    ],
    outputs=gr.Number(label="Predicted IOP (mmHg)"),
    title="Eye IOP Prediction System"
)

app.launch()
```

9. Complete System Flow

Patient Information

|

Gradio Input Components

|

predict_iop() Function

|

Pandas DataFrame

|

Trained XGBoost Regressor

|

Predicted IOP

|

Gradio Displays the Result

10. Key Teaching Points

1. XGBRegressor predicts continuous numerical values.

2. XGBoost uses many trees sequentially, and each new tree tries to correct previous mistakes.

3. fit() trains the model and predict() generates predictions.

4. MAE and RMSE should generally be lower, while R2 should generally be higher.

5. Gradio is not a machine learning algorithm; it is a user-interface library.

6. Gradio connects the user to the Python prediction function.

7. XGBoost is the prediction engine; Gradio is the interface.

11. Quick Revision Questions

1. What type of output is predicted by a regressor?

2. What is the full form of XGBoost?

3. How does boosting improve predictions?

4. What is the purpose of model.fit()?

5. What is the purpose of model.predict()?

6. Which is better for MAE: a lower or higher value?

7. What does Gradio do?

8. What is the difference between gr.Interface() and gr.Blocks()?

Medical-use note: A classroom eye dataset or synthetic dataset is suitable for learning machine learning, but a model should not be used for diagnosis or clinical decisions unless it has been developed and validated under appropriate medical, ethical, and regulatory standards.